

# Python Basics

## Concept Reference Sheet

13 Topics · Definitions · Syntax · Examples · Tips

## 1 Variables & Data Types

*A variable is a named container that stores a value. The type of value it holds is called its data type.*

### Common Data Types

- int — Whole numbers e.g. age = 20
- float — Decimal numbers e.g. gpa = 3.5
- str — Text (in quotes) e.g. name = "Alice"
- bool — True or False e.g. is\_student = True

### Declaring Variables

```
name = "Alice"    # str
age   = 20        # int
gpa   = 3.8       # float
passed = True     # bool
```

### Check & Convert Types

```
print(type(age))    # <class "int">

age_str = str(age)   # int → string
age_int = int("25")  # string → int
price   = float("9.99") # string → float
```

### Multiple Assignment & Swap

```
a, b, c = 10, 20, 30 # assign three at once
a, b     = b, a       # swap values
```

 **Tip:** Variable names cannot start with a number and cannot have spaces. Use underscores: my\_name, not "my name".

## 2 Operators

*Operators are symbols that perform operations on values and variables.*

### Arithmetic Operators

```
x = 17 , y = 5

x + y → 22    # Addition
x - y → 12    # Subtraction
```

```
x * y    → 85      # Multiplication
x / y    → 3.4     # Division      (always float)
x // y   → 3       # Floor division (whole number)
x % y    → 2       # Modulus       (remainder)
x ** y   → 1419857 # Exponentiation (power)
```

## Comparison Operators → return True or False

```
a == b    # Equal to
a != b    # Not equal to
a > b     # Greater than
a < b     # Less than
a >= b    # Greater than or equal
a <= b    # Less than or equal
```

## Logical Operators

```
and → True only if BOTH sides are True
or  → True if AT LEAST ONE side is True
not → Flips True to False, and False to True

age >= 18 and has_id → True only if both are True
age < 18 or has_id  → True if either is True
not True           → False
```

## Assignment Operators

```
x = 10
x += 5    # x = x + 5 → 15
x -= 3    # x = x - 3 → 12
x *= 2    # x = x * 2 → 24
x //= 4   # x = x // 4 → 6
```

 **Tip:** Use % (modulus) to check if a number is even: if num % 2 == 0 means the number is even.

# 3 Input / Output

*input() reads text typed by the user. print() displays output on the screen.*

## print() — Display Output

```
print("Hello World")          # → Hello World
print("Name:", name)          # → Name: Alice
print(f"Hello {name}, age {age}") # → Hello Alice, age 20
print(f"Score: {score:.2f}")   # → Score: 95.68
print("a", "b", sep="-")      # → a-b
print("Hi", end=" ")          # → Hi (no newline at end)
```

## input() — Take User Input

```
name = input("Enter your name: ")    # always returns a string

# Convert if you need a number:
age = int(input("Enter age: "))       # convert to int
price = float(input("Enter price: ")) # convert to float
```

## f-strings — Formatted Output

```
name = "Alice"
score = 95.678

print(f"Student: {name}")          # → Student: Alice
print(f"Score: {score:.2f}")       # → Score: 95.68
print(f"Length: {len(name)}")      # → Length: 5
print(f"Upper: {name.upper()}")    # → Upper: ALICE
```

 **Tip:** `input()` ALWAYS returns a string. If you need a number, wrap it: `int(input(...))` or `float(input(...))`.

# 4 Conditionals

*Conditionals let your program make decisions and run different blocks of code based on whether a condition is True or False.*

## Basic Syntax

```
if condition:
    # runs if condition is True
elif another_condition:
    # runs if the first was False and this is True
else:
    # runs if ALL conditions above were False
```

## Example — Grade Checker

```
score = int(input("Enter score: "))

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
else:
    print("Grade: F")
```

## Nested Conditionals

```
if is_member:
    if age >= 18:
        print("Access granted.")
    else:
        print("Must be 18+ to enter.")
else:
    print("Must be a member.")
```

 **Tip:** Python uses indentation (4 spaces) to define code blocks. Always be consistent — mixing tabs and spaces causes errors.

## 5 Loops

*Loops repeat a block of code multiple times. Use `for` when you know how many times, and `while` when you don't.*

### for Loop

```
for i in range(1, 6):      # 1, 2, 3, 4, 5
    print(i)              # → 1 2 3 4 5 (one per line)

# range(start, stop, step)
for i in range(0, 10, 2): # 0, 2, 4, 6, 8
    print(i)              # → 0 2 4 6 8 (one per line)
```

### while Loop

```
count = 1
while count <= 5:
    print(count)    # → 1 2 3 4 5 (one per line)
    count += 1      # Always update the counter!
```

### break & continue

```
for num in range(1, 11):
    if num == 5:
        continue    # skip 5 – go to next iteration
    if num == 8:
        break        # stop the loop completely
    print(num)       # prints: 1 2 3 4 6 7
```

### Nested Loop — Multiplication Table

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i*j}")

# → 1 x 1 = 1
# → 1 x 2 = 2
# → 1 x 3 = 3
# → 2 x 1 = 2 ... and so on
```

 **Tip:** A while loop that never becomes False will run forever (infinite loop). Always make sure the condition can be reached.

## 6 Lists

*A list stores multiple values in a single variable. Lists are ordered, changeable, and allow duplicate values.*

### Create & Access

```
fruits = ["apple", "banana", "cherry"]

fruits[0]    # "apple"          (first item)
fruits[-1]   # "cherry"         (last item)
```

```
fruits[1:3] # ["banana", "cherry"] (slice)
```

## Common Methods

```
fruits.append("mango") # add to the end
fruits.insert(1, "grape") # insert at index 1
fruits.remove("banana") # remove by value
fruits.pop() # remove last item
fruits.pop(0) # remove first item
fruits.sort() # sort A to Z
fruits.reverse() # reverse the list
fruits.clear() # empty the list
len(fruits) # number of items
```

## Loop Through a List

```
for fruit in fruits:
    print(fruit) # → apple
                # → banana
                # → cherry
```

## Useful Functions

```
numbers = [5, 2, 9, 1, 7]

max(numbers) # 9
min(numbers) # 1
sum(numbers) # 24
sorted(numbers) # [1, 2, 5, 7, 9] (does not change original)
```

 **Tip:** Lists use square brackets `[]`. Indexing starts at 0. Use negative indexes to access from the end: `list[-1]` is the last item.

# 7 Tuples

*A tuple is like a list but immutable — once created, its values cannot be changed. Use tuples for data that should stay fixed.*

## Create & Access

```
colors = ("red", "green", "blue")
point = (10, 20)

colors[0] # "red"
colors[-1] # "blue"
len(colors) # 3
```

## Tuple Unpacking

```
student = ("Alice", 21, "Computer Science")

name, age, major = student
print(name) # Alice
print(age) # 21
print(major) # Computer Science
```

## Tuples are Immutable

```
colors[0] = "yellow"    # ERROR – tuples cannot be changed
```

## Loop Through a Tuple

```
for color in colors:
    print(color)        # → red
                        # → green
                        # → blue
```

 **Tip:** Use tuples for data that should never change — days of the week, coordinates, fixed settings. They are faster than lists.

# 8 Dictionaries

*A dictionary stores data in key-value pairs. Each key is unique and maps to a value — like a real dictionary where a word (key) has a definition (value).*

## Create & Access

```
student = {
    "name" : "Alice",
    "age"  : 21,
    "grade": "A"
}

student["name"]    # "Alice"
student.get("age") # 21 — safe, no error if key missing
```

## Add, Update & Delete

```
student["email"] = "a@email.com" # add a new key
student["age"]    = 22            # update existing key
del student["email"]             # delete a key
```

## Loop Through a Dictionary

```
for key, value in student.items():
    print(f"{key}: {value}")    # → name: Alice
                                # → age: 21
                                # → grade: A

# Keys only:    for key in student.keys()
# Values only:  for val in student.values()
```

## Useful Methods

```
student.keys()    # all keys
student.values()  # all values
student.items()   # all key-value pairs
len(student)      # number of key-value pairs
```

 **Tip:** Use `.get(key)` instead of `[key]` when unsure if a key exists — it returns `None` instead of an error. e.g. `student.get("phone", "N/A")`

## 9 Sets

*A set stores a collection of unique values with no duplicates and no fixed order. Useful for removing duplicates and comparing groups of data.*

### Create a Set

```
numbers = {1, 2, 3, 4, 5}
dups    = {1, 2, 2, 3, 3} # stored as {1, 2, 3} — no duplicates
empty   = set()          # empty set (not {} which is a dict)
```

### Add & Remove

```
numbers.add(6)      # add a value
numbers.discard(3)  # remove — no error if not found
numbers.remove(3)   # remove — ERROR if not found
```

### Set Operations

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

a | b    # Union      → {1, 2, 3, 4, 5, 6} (all items)
a & b    # Intersection → {3, 4}          (common items)
a - b    # Difference  → {1, 2}          (in a but not b)
```

### Remove Duplicates from a List

```
names = ["Alice", "Bob", "Alice", "Charlie", "Bob"]
unique = list(set(names))
print(unique) # ["Alice", "Bob", "Charlie"]
```

 **Tip:** Sets use {} like dictionaries but have no keys. An empty set **MUST** be created with set(), not {} — {} creates an empty dictionary.

## 10 Strings

*A string is a sequence of characters enclosed in quotes. Strings are immutable — methods return a new string, they do not change the original.*

### Creating Strings

```
name      = "Alice"
message   = 'Hello World'
para      = """This is a
multi-line string"""
```

### Accessing Characters & Slicing

```
word = "programming"
```

```
word[0]      # "p"           first character
word[-1]     # "g"           last character
word[0:6]    # "progra"      slice
word[::-1]   # "gnimmargorp" reversed
```

## Common String Methods

```
text = " Hello World "
```

```
text.strip()           # "Hello World"
text.lower()           # " hello world "
text.upper()           # " HELLO WORLD "
text.title()           # " Hello World "
text.replace("World", "Python") # " Hello Python "
text.strip().split()   # ["Hello", "World"]
" - ".join(["a", "b", "c"]) # "a - b - c"
"hello".count("l")     # 2
"hello".find("e")      # 1 (index of first match)
```

## Checking String Content

```
"hello".startswith("he") # True
"hello".endswith("lo")  # True
"hello123".isalnum()    # True (letters and numbers only)
"hello".isalpha()       # True (letters only)
"123".isdigit()         # True (digits only)
```

## f-strings

```
name = "Alice"
score = 88.5
```

```
print(f"Name: {name}")      # → Name: Alice
print(f"Score: {score:.2f}") # → Score: 88.50
print(f"Upper: {name.upper()}") # → Upper: ALICE
```

 **Tip:** Strings are immutable. `text.upper()` does NOT change `text` — it returns a NEW string. To save it: `text = text.upper()`

# 11 Built-in Functions (no import needed)

*Python includes many powerful functions ready to use without any import. These are the most frequently needed ones.*

## Type & Conversion

```
type(x)      # returns the data type of x
int("25")    # "25" → 25      convert to integer
float("3.14") # "3.14" → 3.14  convert to float
str(100)     # 100 → "100"    convert to string
bool(0)      # 0 → False
bool(1)      # 1 → True
```

## Numeric Functions

```
abs(-7)      # absolute value → 7
```



```

round(3.567, 2)    # round to 2 decimals    → 3.57
pow(2, 3)          # 2 to the power 3       → 8
max(3, 7, 1)       # largest value          → 7
min(3, 7, 1)       # smallest value         → 1
sum([1,2,3,4])     # sum of a list          → 10

```

## Sequence Functions

```

len("hello")       # length of string/list/etc → 5
range(1, 6)        # generates 1 2 3 4 5
range(0, 10, 2)    # generates 0 2 4 6 8      (step 2)
sorted([3,1,2])    # sorted list → [1, 2, 3]   (original unchanged)
sorted([3,1,2], reverse=True) # → [3, 2, 1]
list(range(1, 6))  # [1, 2, 3, 4, 5]

```

## enumerate() — index + value in a loop

```

fruits = ["apple", "banana", "cherry"]

for i, fruit in enumerate(fruits):
    print(f"{i+1}. {fruit}")

# Output:
# 1. apple
# 2. banana
# 3. cherry

```

 **Tip:** range(stop), range(start, stop), and range(start, stop, step) are all valid. The stop value is never included.

# 12 math Module → import math

*The math module gives access to mathematical functions and constants. Always import it at the top of your program.*

## Import

```
import math
```

## Common Functions

```

math.sqrt(16)      # square root          → 4.0
math.pow(2, 10)    # 2 to the power 10     → 1024.0
math.floor(4.9)    # round DOWN            → 4
math.ceil(4.1)     # round UP              → 5
math.factorial(5)  # 5! = 5×4×3×2×1        → 120
math.log(100, 10)  # log base 10 of 100    → 2.0
math.log2(8)       # log base 2 of 8       → 3.0

```

## Constants

```

math.pi           # 3.141592653589793
math.e             # 2.718281828459045   (Euler's number)
math.inf           # infinity

```

## Example — Circle Area

```
import math

radius = float(input("Enter radius: ")) # e.g. user enters 5
area = math.pi * math.pow(radius, 2)
print(f"Area: {area:.2f}") # → Area: 78.54
```

 **Tip:** `math.pow()` always returns a float. Use `**` for integer results: `2**3` gives 8 (int), `math.pow(2, 3)` gives 8.0 (float).

## 13 random Module → import random

*The random module generates random numbers and makes random selections — useful for games, simulations, and testing.*

### Import

```
import random
```

### Common Functions

```
random.randint(1, 10) # random integer from 1 to 10 (inclusive)
random.random() # random float between 0.0 and 1.0
random.uniform(1.5, 5.5) # random float between 1.5 and 5.5
random.choice(["a", "b", "c"]) # pick one item randomly from a list
random.shuffle(my_list) # shuffle list in place (changes original)
random.sample(my_list, 3) # pick 3 unique random items (new list)
```

## Example — Dice Roller

```
import random

roll = random.randint(1, 6)
print(f"You rolled: {roll}") # → You rolled: 4 (any number 1-6)
```

## Example — Random Item Picker

```
import random

questions = ["What is 2+2?", "Capital of France?", "What is H2O?"]
picked = random.choice(questions)
print(f"Question: {picked}") # → Question: Capital of France? (random)
```

 **Tip:** `random.shuffle()` modifies the original list in place. `random.sample()` leaves the original list unchanged and returns a new list.

## Quick Reference Summary

| Topic     | Created With              | Mutable? | Key Feature                |
|-----------|---------------------------|----------|----------------------------|
| Variables | <code>name = value</code> | Yes      | Stores a single value      |
| List      | <code>[ ]</code>          | Yes      | Ordered, allows duplicates |
| Tuple     | <code>( )</code>          | No       | Ordered, fixed values      |

|            |                |     |                              |
|------------|----------------|-----|------------------------------|
| Dictionary | { key: value } | Yes | Key-value pairs              |
| Set        | { } or set()   | Yes | Unique values, no order      |
| String     | " " or ' '     | No  | Text, sequence of characters |

## Common Functions Quick Lookup

| Function            | Usage                    | Returns         |
|---------------------|--------------------------|-----------------|
| len(x)              | len([1,2,3])             | 3               |
| range(s,e)          | range(1,6)               | 1 2 3 4 5       |
| type(x)             | type(42)                 | <class 'int'>   |
| abs(x)              | abs(-7)                  | 7               |
| round(x,d)          | round(3.567,2)           | 3.57            |
| max(x)              | max([3,7,1])             | 7               |
| min(x)              | min([3,7,1])             | 1               |
| sum(x)              | sum([1,2,3])             | 6               |
| sorted(x)           | sorted([3,1,2])          | [1,2,3]         |
| enumerate(x)        | enumerate(['a','b'])     | (0,'a'),(1,'b') |
| math.sqrt(x)        | math.sqrt(16)            | 4.0             |
| math.pow(x,y)       | math.pow(2,8)            | 256.0           |
| math.floor(x)       | math.floor(4.9)          | 4               |
| math.ceil(x)        | math.ceil(4.1)           | 5               |
| random.randint(a,b) | random.randint(1,6)      | e.g. 4          |
| random.choice(lst)  | random.choice(['a','b']) | e.g. 'b'        |